

Foundations and the AI service layer

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build `nw/llm`: the one client every later project calls a model through. Bounded concurrency, provider-aware retries, validated structured output, ordered fan-out, a hard spend cap, and a correlation ID on every line.

Where we are

PART 1

Foundations and
the AI service layer

PART 2

Machine learning in
production

PART 3

Deep learning and
representation

PART 4

Generative AI:
retrieval and
evaluation

PART 5

Agentic AI systems

PART 6

Deployment,
observability and
capstone

One domain for six parts: Northwind's support desk. Each project becomes a tool for the next one, and the capstone puts all of them on your cloud.

Today, step by step

- 1 Confirm the machine and the track
- 2 Read the contract and see the tests fail
- 3 Backoff with jitter, honouring `Retry-After`
- 4 Complete with a semaphore, retries and metering
- 5 Structured output with one repair round trip
- 6 Fan out and keep the order
- 7 Install the skill set and set the review convention
- 8 One live call on your track

How the room works

Every step ends in a check you tick in the guide. Red tests are the spec; green is the proof.

Your coding agent writes alongside you from step 7 on. Same bar: it passes the tests or it does not go in.

Every project hits the same three walls

Fan-out

500 prompts, 500 open connections, then a wall of 429s.

Bounded concurrency.

Unreliable upstream

The batch dies at prompt 312 and everything before it is lost.

Provider-aware retries.

Unvalidated output

`json.loads` raises on a reply that begins "Sure, here is".

Structured output with repair.

Solve them once, in one client. Every project imports the result and never opens a connection to a model any other way.

A model call is waiting, not computing

Hundreds of milliseconds to several seconds on the network, almost no local work.

That single fact decides the design:

- `asyncio`, not threads: idle waiting costs nothing
- one `httpx` pool per process
- a semaphore sized to the provider's limit

The semaphore is a business number

It comes from the provider's quota and from how many services share the account, not from how many cores the box has.

Two services, one quota of 50 a minute: neither gets 50.

Retryable and terminal are different failures

Status or error	Family	Why
429, 529, 5xx, timeout	retryable	the provider's or the network's problem, and transient
400, 401, 403, 404	terminal	your request; it fails the same way again
refusal stop reason	terminal	the content, not the call

Retrying a 400 spends money on a certain failure. Not retrying a 429 loses work that would have succeeded a second later.

When retries are not enough: fall back, then stop asking

- **Fallback:** `NW_MODEL_FALLBACK_<ROLE>` names a second model per role; tried once when the primary is exhausted or gone, never for a refusal or a bad request that is ours
- **Circuit breaker:** after `NW_BREAKER_FAILURES` failures in a row a model is skipped for `NW_BREAKER_OPEN_S` seconds, so a dead model costs no retries
- **Timeout:** every call runs under `NW_REQUEST_TIMEOUT_S`; a provider that never answers becomes a retryable timeout

Every completion says how it was obtained

The span and the log line carry the model that answered, `fallback`, `attempts` and the models the breaker skipped. The cost record names the fallback model, not the one you asked for.

Backoff needs jitter, or the spike comes back

```
ceiling = min(base * 2 ** (attempt - 1), cap)
delay = random.uniform(0, ceiling)
if retry_after is not None:
    delay = retry_after
```

- The ceiling doubles per attempt, the delay is uniform below it
- Without jitter, every client that failed together retries together, and the provider sees the same spike again
- `Retry-After` overrides everything: the provider chose that number

Structured output is a contract, not a hope

- 1 Tell the model the JSON Schema of the Pydantic class you want
- 2 Parse, then validate. Never trust the formatting
- 3 On failure, send the error back once, with the reply it made
- 4 Second failure: raise. A loop that keeps asking is a budget leak

One repair fixes most malformed replies in practice. Vendor JSON modes are faster on some models and are also the first thing that breaks on a vendor change; validation in your own code survives it.

Cost is an output of every call

0.00032 USD

three round trips in preflight, priced from the table

- Every completion is priced from `nw/llm/prices.py` and recorded with its correlation ID
- The meter **reserves** an estimate before the call and replaces it with the real cost after
- The cap is a hard stop that refuses the call, not a dashboard that notices later
- The capstone turns these records into cost per resolved ticket

Code names roles, configuration names models

Role	Who calls it	Today
Workhorse	the service layer, retrieval, the agent, the capstone	Claude Sonnet 5
Judge	the evaluation harness	Claude Opus 5
Economy	the capstone router	Claude Haiku 4.5

`ModelRole.WORKHORSE` in code; the ID in `.env`. Swapping a vendor is one provider file and one config change, and nothing outside `nw/llm/providers/` imports a vendor SDK.

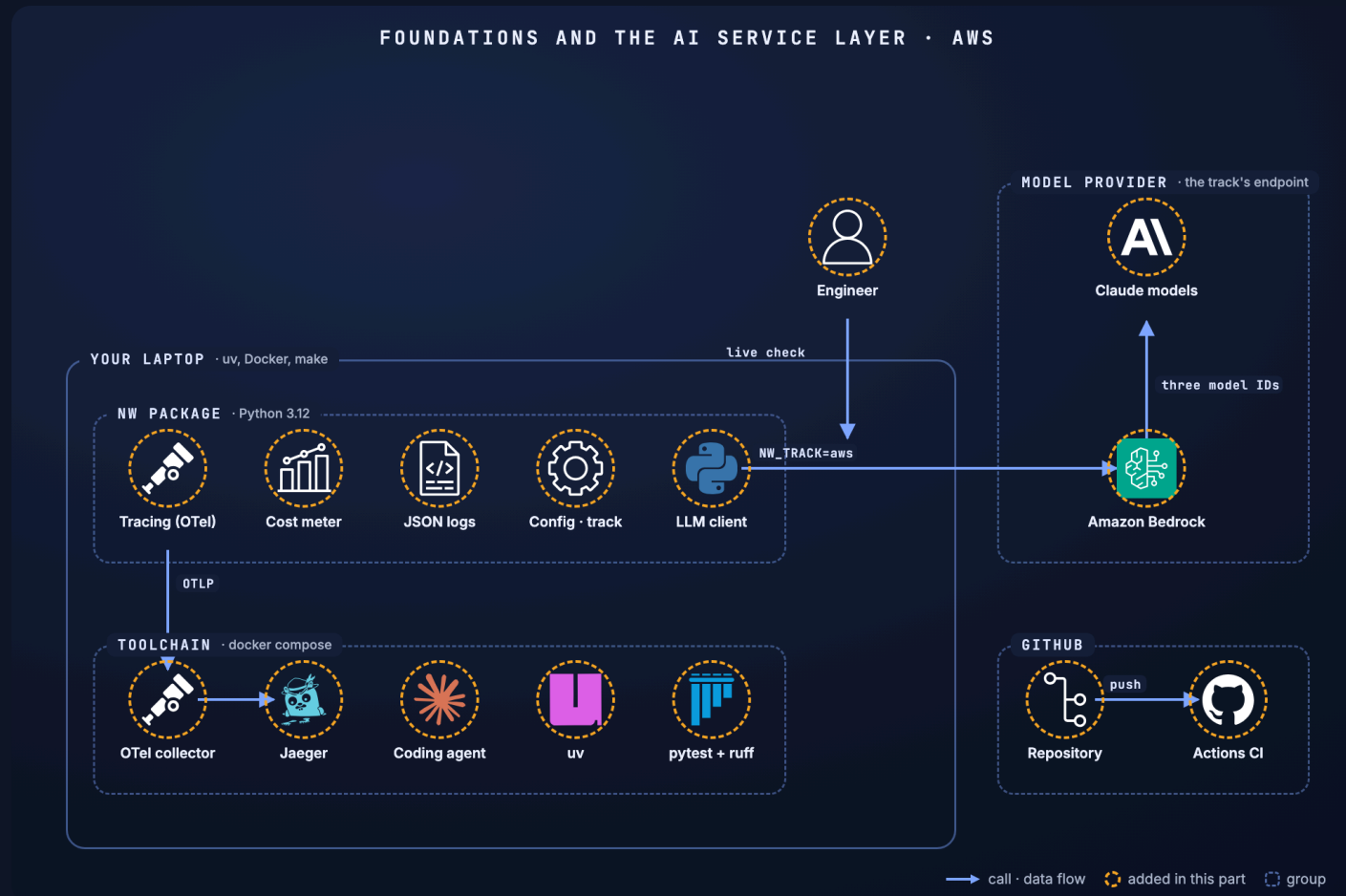
The correlation ID answers "what happened to my ticket"

- One context variable, set once per request
- Follows the async task through every `await` without being passed around
- On every log line, every cost record, every trace span

Trace, log, bill: one key

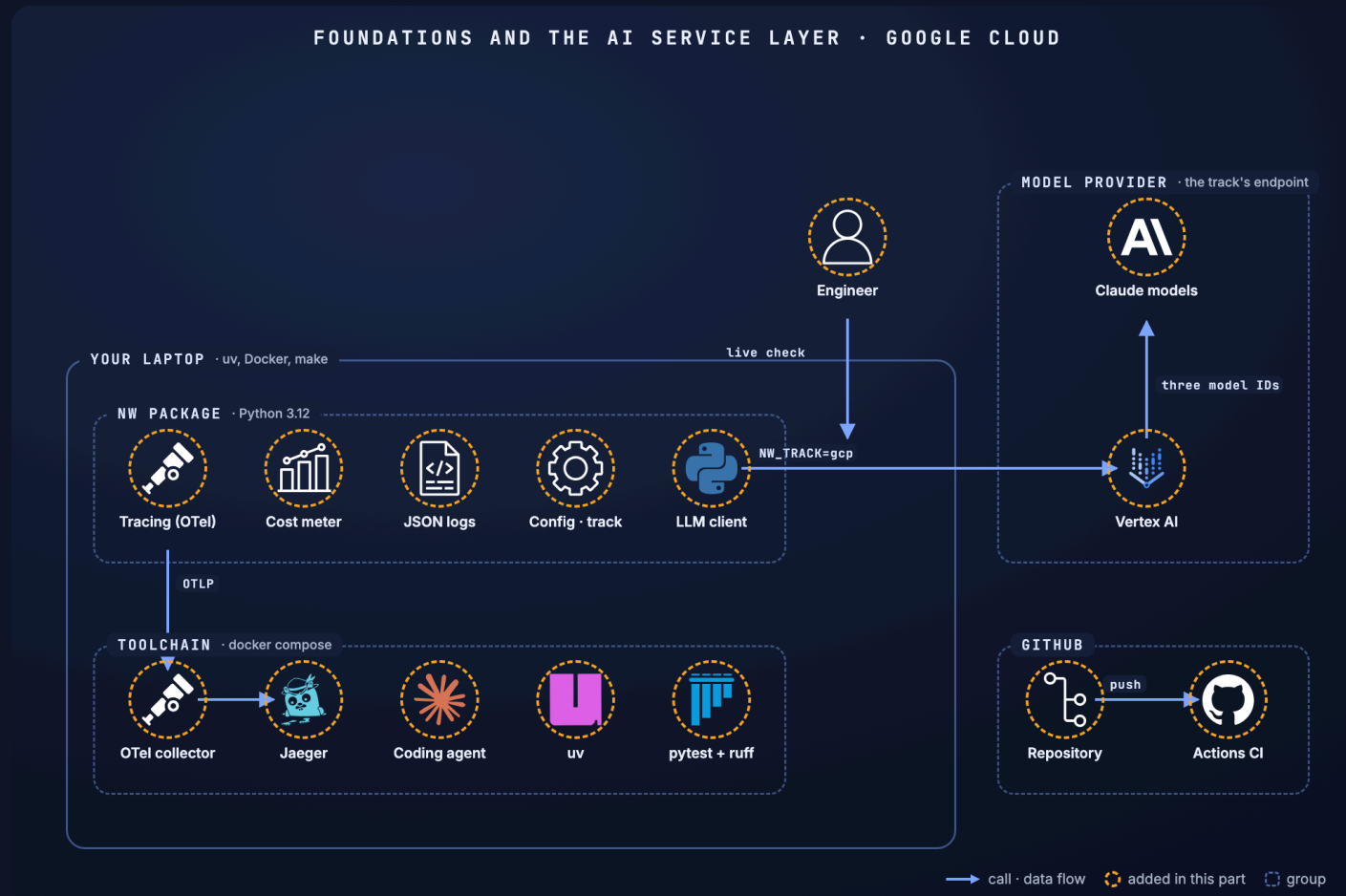
When support asks what happened to ticket T-200005, you search one ID and get the model calls, their cost and the spans in the order they ran.

What exists after today, AWS track



The nw package on your laptop, the local toolchain, the track's model endpoint, and the repository with CI. Ringed: what this part adds.

What exists after today, Google Cloud track



Same package, same toolchain; Vertex AI is the endpoint. The provider file is the only difference.

Agent-written code, same bar

`/tdd`

Start from the failing test, not from the prompt.

`/code-review`

Before you run the suite, not after it is green.

`/session-check`

Reads the acceptance criteria against your checkout and reports each one.

Nothing enters a project because an agent produced it. It passes the tests or it does not go in.

Acceptance criteria

1. 100 prompts at `max_concurrency=8` never exceed 8 in flight
2. Two 429s then success: succeeds. A 400: fails at once, one attempt
3. `structured` recovers from one malformed reply and raises after the second
4. `map` preserves input order under out-of-order replies
5. Spend and tokens are queryable and the cap is a hard stop

85 passed

what `make session01` prints when you are done

Questions worth asking

- What is the right concurrency for a provider that allows 50 requests a minute, shared by two services?
- When is the repair round trip more expensive than failing fast?
- Which of today's failures would have reached production in your current setup?